쩌는 게임 기획서, 이렇게 쓴다

How to Write a Great Design Document 역자주

Damion Schubert
Bioware Austin

www.zenofdesign.com

강연자 소개
(About Me)

- 10년 경력의 MMO 게임 디자이너.
  - 대부분 경우, 리드 게임 디자이너의 역할을 수행.
- 매우 복잡한 시스템하에서 개발.
- 좋은 문서화 프로세스를 반기게 됨.
- '문서화 전문가(doc guy)'가 되는 것이 자신의 경력에 도움이 된다는 사실을 발견.
- 아직도 어떻게 해야 제대로 하는 것인지에 대해서 배우고 있는 중.

기획서에 대한 편견
(What I hear)

- “기획서 작성은 시간 낭비야.”
- “아무도 기획서를 읽지 않는다고.”
- “우리 프로그래머들은 기획서 검토가 노가다나 다름 없다고 하더군.”
- 이런 이야기들은 문서화에 대한 일반론이 아니라, 여러분이 작성한 문서에 대한 평판입니다.

잔혹한 사실들
(The Harsh Truth)

- 분명히 모든 게임 디자이너들은 자신들의 생각을 다른 사람들과 공유하고 싶어합니다.
- 모든 프로그래머들(과 다른 팀원들)은 분명 자신들이 만들고 있는 게 무언지 알고 싶어합니다.

- 그럼에도 불구하고, 대부분의 기획서는 그리 훌륭하지 못하며, 대부분의 문서화 프로세스는 재미를 발견해가는 게임 개발의 반복적인 속성을 무시하고 있습니다.

이 강연의 내용
(This Talk)

- 이 강연의 목표는 무엇인가?
- 왜 좋은 기획서를 찾기 힘든가?
- 어떤 규칙에 따라 게임 기획서를 작성하는 게 좋은가?
- 리드들(Leads)역자주이 게임 기획서 문서화 절차를 마련할 때, 어떤 점들을 고려해야 하는가?

강연의 초점:
(Presentation Focus)

시스템 기획 문서(Systems Design Document)

다음은 제외:

- 임원용 요약문 혹은 비전 문서
  (Executive Summaries/Vision Documents)역자주
- 기획서 총람 혹은 상세 기획 검토
  (Design Overview Documents/DDRs)
- 테스트 계획(Test Plans)

좋은 기획서의 목표
(Goals of Good Docs)

- 기획에 대한 합의를 담고 있습니다.
- 부서간 핵심 비전을 공유하는 통로가 됩니다.
- 일정 관리를 손쉽게 해줍니다.
- 개발에 촛점(focus)을 제공합니다.
- 향후 업무간의 연관관계 및 기획상의 상충되는 부분들을 미리 확인할 수 있게 해줍니다.

좋은 기획서는 왜 그렇게 찾기 힘들까?
(Why is good design documentation so rare?)

- 기획서들이 너무 복잡하게 서로 연결된 시스템들을 다룹니다.
- 게임 디자이너들이 너무 많이 기획하는 경향이 있습니다.
  - 시스템을 구축하는 것보다, 기획하는 게 시간이 더 적게 걸립니다.
  - "바보 대전(Big Book of Stupid)"
- 기획서들이 점진적이고 반복적인 개발을 포용하지 않습니다.
- 프로젝트 진행에 따른 변화가 기획서에 거의 반영되지 않습니다.

좋은 기획서 작성을 위한 규칙들
(Rules of good Design Documentation)

다른 개발자들 왈,
(What other devs say:)

"그저 짧고, 목표가 분명한 최신 문서를 좀 건내줘."

"짧고, 정확하고, 코딩할 부분을 쉽게 찾을 수 있는".

"난 그냥 내가 할 일들만 죽
나열된 목록을 원해."

1. 독자를 파악하라
(Know your Target)

- 기획서에 관심이 있는 사람들:
  - **기획팀.** 기획에 대해 합의하려고.
  - **프로그래밍팀.** 게임을 구현하려고.
  - **프로듀서들.** 일정을 세우고, 예산을 할당받으려고.
  - **품질보증 부서 (QA).** 테스트 계획을 수립하려고.
  - **외부 파트너들.** 개발팀을 귀찮게 하려고.

1. 독자를 파악하라
(Know your Target)

- **프로그래머들**이 가장 중요한 독자입니다.
  - 실제 게임을 구현하는 사람들이기 때문이죠.
  - 또한 다른 독자들에게는 대체로 다른 문서들이 더 유용하기 때문이기도 합니다.
- 프로그래머들에게 무엇이 필요한지 물어보세요.
  - 만약 프로그래머들이 제 규칙들 중 하나를 무시하라고 한다면, 그렇게 하셔도 무방합니다.

2. 짧게 써라
(Keep it Short)

짧은 문서는:

- 읽기 더 쉽고,
- 쓰기 더 쉽고,
- 관리하고 더 편하고,
- 정치적으로 다루기가 더 용이하고,
- 모순될 가능성이 더 적고,
- 단순한 기획이 될 가능성이 더 높습니다.

2. 짧게 써라
(Keep it Short)

- 군더더기(the fluff)를 제거하세요.
- 빈 항목들을 제거하세요.
- ‘복사해서 붙여넣기’를 피하세요.
- 그 자체로 분명한 시스템에 불필요하게 덧붙인 설명을 제거하세요.

2. 짧게 써라
(Keep it Short)

너무 깁니다

길드 초대 확인 UI. 플레이어가 길드를 생성하면 확인 UI가 나타납니다. 확인 UI는 “이 길드에 가입하시겠습니까?”라고 묻고, ‘확인’ 버튼과 ‘취소’ 버튼이 달려 있습니다.

확인 버튼. 확인 UI에는 확인 버튼이 달려 있고, 이 동작을 승인합니다.

취소 버튼. 확인 UI에는 취소 버튼이 달려 있고, 길드 생성을 방지합니다.

닫기 버튼. 확인 UI의 오른쪽 상단에는 ‘X’ 버튼이 달려 있고, 취소 버튼과 동일한 역할을 합니다.

Esc. 이스케이프를 누르면 이 동작을 취소하고, 취소 버튼을 누르는 것과 같습니다.

2. 짧게 써라
(Keep it Short)

이게 더 좋습니다

길드 초대 확인 UI.

플레이어들이 길드에 초대를 받으면, 확인창이 열립니다 (일반 대화창.doc 참조).

2. 짧게 써라
(Keep it Short)

누가 신경이나 쓰나요?

아이템 제작세. 대장장이의 신, 헤파이스토스는 아테네의 장인들에게 자신의 뜻을 가르쳤고, 모든 장인들은 그의 위대함을 경배합니다. 그런 의미에서 아이템을 제작하고자 플레이어라면 누구나 그의 은혜를 입기 위해서 헤파이토스의 신전에 십일조를 바쳐야 합니다. 단, 신들의 해머와 같은 아이템을 갖고 있으면, 제작세를 면제받을 수 있습니다.

2. 짧게 써라
(Keep it Short)

이게 더 좋습니다

아이템 제작세. 아이템을 제작할 때마다, 플레이어는 해당 지역의 신전에 십일조를 납부해야 합니다.

제작세 면제. 플레이어들이 어떤 아이템들을 소지하고 있으

2. 짧게 써라
(Keep it Short)

- 명심하세요:
  - 프로그래머들은 거의 언제나 짧은 목록을 좋아한답니다.
  - (프로그래머들은 목록에 있는 것들을 지워나가는 걸 좋아한다고 할 수 있죠.)

3. 우선순위를 부여하라
(Prioritize the Design)

- 피처역자주에 우선순위를 부여하고, 단계별로 쪼갭니다.
- 문서에 후반부 피처를 명백히 분리해놓으세요.

3. 우선순위를 부여하라
(Prioritize the Design)

구려요!

플레이어는 소지품 창에서 아이템을 장착할 수 있습니다.
플레이어가 아이템을 장착하면, 전투력이 변경됩니다.
플레이어가 장비를 입으면, 겉으로 표시됩니다.
플레이어들의 장비는 특수 효과가 부여될 수도 있습니다.
플레이어는 자신의 방패에 자신이 속한 길드의 문장을
그려넣을 수 있습니다.

3. 우선순위를 부여하라
(Prioritize the Design)

아직도 별로입니다

(1단계) 플레이어는 소지품 창에서 아이템을 장착할 수 있습니다.

(1단계) 플레이어가 아이템을 장착하면, 전투력이 변경됩니다.
(2단계) 플레이어가 장비를 입으면, 겉으로 표시됩니다.
(2단계) 플레이어들의 장비는 특수 효과가 부여될 수도 있습니다.

(4단계) 플레이어는 자신의 방패에 자신이 속한 길드의 문장을 그려넣을 수 있습니다.

3. 우선순위를 부여하라
(Prioritize the Design)

이게 더 좋습니다

기본 장비 **(프로토타입)**

플레이어는 소지품 창에서 아이템을 장착할 수 있습니다.
플레이어가 아이템을 장착하면, 전투력이 변경됩니다.

고급 장비 **(2단계)**

플레이어가 장비를 입으면, 겉으로 표시됩니다.
플레이어들의 장비는 특수 효과가 부여될 수도 있습니다.

장비와 길드 문장 **(4단계)**

플레이어는 자신의 방패에 자신이 속한 길드의 문장을 그려넣을 수 있습니다.

3. 우선순위를 부여하라
(Prioritize the Design)

- 1단계: 피처를 프로토타이핑합니다.
  - (게임 아이디어를 검증/시연하는 게 필수.)
- 2단계: 핵심 피처를 구현합니다.
  - (컨텐트 생성과 관련된 피처와 도구들이 여기에 해당합니다.)
- 3단계: 출시 시점에 반드시 들어갈 것들.
  - (2순위 피처들에 관련된 피처들이 포함)
- 4단계: 희망 사항, 아마도 확장팩.
- 5단계: 야~ 신난다~

3. 우선순위를 부여하라
(Prioritize the Design)

- 우선순위 결정은 단순히 각각의 피처에 대한 것이 아니라, 프로젝트 전체에 대한 것입니다 - 전체 피처나 기획서는 2단계, 3단계 혹은 4단계의 피처들로 구성되어 있을 수도 있습니다.

4. 보여주어라
(Illustrate)

- 백문이 불여일견.
- 요령:
  - 유사한 피처들을 가진 다른 게임들의 스크린샷
  - Visio 다이어그램
  - '예제' 텍스트

4. 보여주어라
(Illustrate)

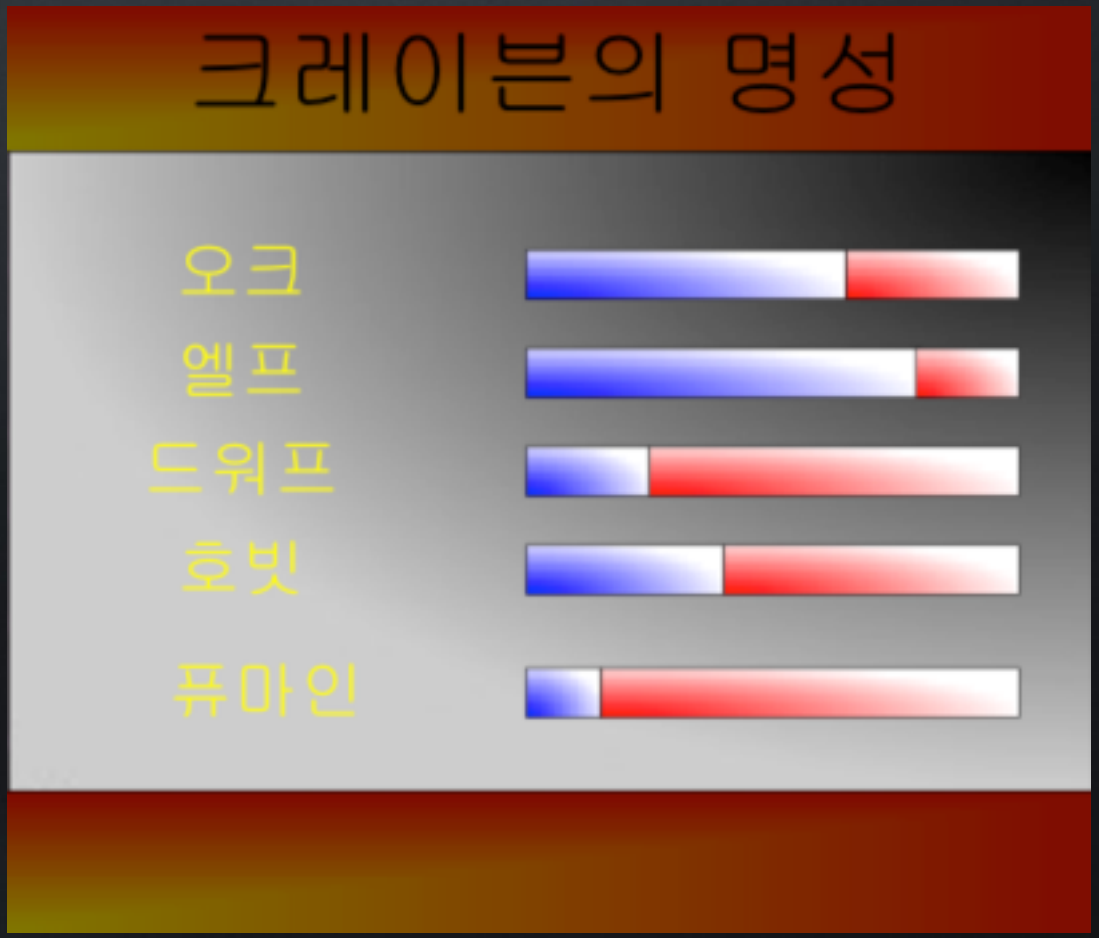
예제

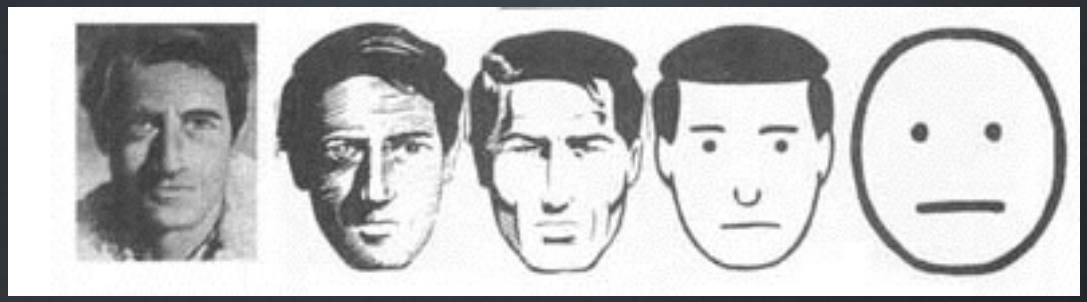
플레이어는 자신의 기술표에서 기술을 삭제할 수 있으며, 특수 NPC(the 'mindwiper')를 찾아가서 지우고 싶은 기술을 선택하면 됩니다.
기술 삭제에는 금전적인 비용이 듭니다.
다른 기술의 전제 조건이 되는 기술은 삭제할 수 없습니다.

조 밥은 기초 초능력과 고급 초능력을 잊어버리기로 결정했고, mindwiper를 찾아갔습니다. 그는 기초 초능력을 삭제하려고 했지만 실패했는데, 그 이유는 고급 초능력의 전제 조건이었기 때문입니다. 그래서 이번에는 고급 초능력을 삭제한 후, 기초 초능력을 삭제했고, 둘 다 문제 없이 삭제되었습니다.

4. 보여주어라
(Illustrate)

여기서 무엇이 잘못 되었을까요?





- 그림이 더 추상적일수록, 독자가 자신의 관점을 투사하기가 더 쉬워집니다.
- UI 아티스트에게 기능적으로 정확한 것을 전해주고 싶겠지만, UI 아티스트가 미학적인 부분을 결정하도록 놔두어야 합니다.

5. 다른 사람이 어떻게 작업해야 하는지 대해 시시콜콜 적지 말라
(Don’t tell others How to do their jobs)

믿기지 않을 수도 있지만, 이게 더 좋습니다.

5. 다른 사람이 어떻게 작업해야 하는지 대해 시시콜콜 적지 말라
(Don’t tell others How to do their jobs)

이건 여러분이 상관할 문제가 아닙니다!

“퀘스트.doc”

퀘스트 변수는 캐릭터 오브젝트에 있는 비트 벡터의 연결된 목록에 저장됩니다.

5. 다른 사람이 어떻게 작업해야 하는지 대해 시시콜콜 적지 말라
(Don't tell others How to do their jobs)

이게 여러분이 할 일입니다.

나머지는 코더들이 해결하도록 하세요.

"퀘스트.doc"

퀘스트 변수와 메모리에 대해서 고려할 사항들.

게임에는 약 2,500개의 퀘스트들이 있습니다.

플레이어들은 진행중인 퀘스트를 한 번에 최대 20개까지 갖고 있을 수 있습니다.

플레이어들은 한 퀘스트에서 최대 10번의 선택을 하게 되며, 각 선택지에서 한 결정은 퀘스트가 끝날 때까지 저장되어 있어야 합니다.

나중에 어떤 퀘스트를 완료했느냐에 따라서 컨텐트의 잠금이 해제될 수 있으므로, 각 퀘스트의 완료 상태와 그 결과가 저장되어야만 합니다.

6. 사용자 스토리를 활용하라
(Use user stories)

스크럼의 사용자 스토리 규칙(약자 INVEST를 기억하세요)

- **독립적인(I**ndependent) - 다른 사용자 스토리들과 겹치지 않는.
- **절충가능한(N**egotiable) - 세부사항이나 구현은 최종 사용자의 만족에 비하면 덜 중요함.
- **사용자에게 가치있는(V**aluable) - 최종 사용자의 시각에서 쓰여진.
- **추정가능한(E**stimatable) - 프로그래머가 구조를 설계하고(architect) 일정을 잡을 수 있을정도로는 상세한.
- **작은(S**mall) - 1주일 이상 걸리지 않는.
- **검증가능한(T**estable) - 기획팀과 프로그래밍팀이 함께 완료여부를 확인하고 동의가능한.

6. 사용자 스토리를 활용하라
(Use user stories)

플레이어의 레벨이 상승할 때, 플레이어는 효과음을 듣는다.

너무 짧다!

플레이어는 새로운 우주 대사(ambassador)를 선출할 수 있다.

너무 의미가 넓어!

플레이어의 레벨이 상승하면, 플레이어는 효과음을 듣고, 파티클 효과를 보고, 3점의 속성 점수를 얻고, 5점의 기술 점수를 얻으며, 10레벨일 경우 특등석을 이용할 수 있게 된다.

너무 길다구!

6. 사용자 스토리를 활용하라
(Use user stories)

저희는 사용자 스토리 1개에 몇개의 세부 요구사항을 추가해서 사용하며,
프로그래머가 대략 2~5일 정도 작업할 분량입니다.

경험치가 해당 레벨의 한도를 넘으면, 플레이어는 레벨이 상승합니다.

플레이어는 '딩'하는 효과음을 듣습니다.
플레이어는 파티클 효과를 봅니다.
플레이어는 자신의 능력치를 상승시킬 수 있는 속성점수 5점을 얻습니다.
플레이어는 기술에 투자할 수 있는 기술 점수 3점을 얻습니다.
만약 플레이어가 10 레벨에 도달하면, 특등석을 사용할 수 있게 됩니다(특등석.doc 참조)

모든 문장이 (개발자가 아닌) "플레이어"로 시작한다는 점을 기억하세요.

7. 콘텐트로부터 코드를 분리하라
(Separate Code from Content)

공포의 골머리 기호!

제작 도구. 어떤 제작 기술은 도구가 필요하며, 없을 경우에는 그 기술을 사용할 수 없다는 메시지가 출력됩니다.

- **제련.** 제련에는 대장장이의 망치와 부젓가락이 필요합니다. 고급 망치와 부젓가락이 있으면, 더 다양한 아이템을 만들 수 있습니다.
- **재단.** 재단사가 되려면 베틀이 필요합니다.
- **연금술.** 연금술에는 시험관 세트가 필요합니다. 고급 시험관 세트를 갖고 있으면, 더 다양한 아이템을 만들 수 있습니다.
- **조각.** 조각에는 망치와 끌이 필요합니다.

7. 콘텐트로부터 코드를 분리하라
(Separate Code from Content)

요구사항을 딱 2가지로 - 쉽죠잉!

제작 도구. 어떤 제작 기술은 도구가 필요하며, 없을 경우에는 그 기술을 사용할 수 없다는 메시지가 출력됩니다.

고급 도구. 어떤 제작 기술은 더 강력한 도구를 사용해서 더 강력아 아이템을 제작할 수 있습니다.

제작 기술과 도구		
기술	도구	고급 도구
제련	망치와 부젓가락	예
재단	베틀	
연금술	시험관 세트	예
조각	망치와 끌	

7. 콘텐트로부터 코드를 분리하라
(Separate Code from Content)

- 사람들이 필요한 정보를 찾아 헤매게 만들지 마세요.
- 컨텐트는 부록이나 표로 분리하세요.

8. 좋은 서식에 시간을 투자하라
(Invest in a good Format)

- 팀 공용 서식을 사용하세요.
- 보기 좋은 서체로 변경하세요.
- 가로 구분선을 사용하세요.
- 예제에는 설명선(callout) 상자역자주를 사용하세요.
- 글머리 기호를 사용하세요.
- **그러나** 서식의 노예가 되진 마세요.

뭐가 다른지 아시겠어요?

(Viva la Difference)

- 이것은 Microsoft 파워포인트의 기본 서식입니다
  - 그다지 예쁘지 않죠, 그렇지 않나요?
  - 약간의 시간을 들여 기본 서체를 바꾸거나, 워터마크를 더하는 것만으로도 여러분의 문서가 전문가 삘이 나는데 큰 도움이 될 겁니다.

9. 명확한 용어를 사용하라
(Use Clear Terminology)

독자가 이미 알고 있을 거라고 전제하지 마세요!

이 주문은 높은 DPS를 갖고 있지만, 증오 감소 효과를 갖고 있어서 레이드에서 어그로를 감소시킵니다.

Superchunk 하나당 최대 6명의 spawn agent 만 있을 수 있습니다.

8. 명확한 용어를 사용하라
(Use Clear Terminology)

- 평범하고 쉬운 어휘를 사용하세요.
- 오덕 용어는 피해주세요.
- 회사 내부 용어도 삼가하세요.
- 새로운 용어를 사용할 때는 일관적으로 사용하세요.
- 용어집(glossary)을 만드는 것도 고려해보세요.

10. 중복을 제거하라
(Kill Redundancy)

- 중복은 악이며, 혼란을 야기하고, 오류를 발생시킵니다

중복된 부분을 제거하세요!

"전투 능력치.doc"

근력은 플레이어의 공격력을 근력/2만큼 증가시킵니다.
민첩성은 플레이어의 정확도를 민첩성/3만큼 증가시킵니다.
체취는 플레이어가 NPC를 유학할 확률을 체취/2만큼 감소시킵니다

"아이템.doc"

근력은 플레이어의 공격력을 근력/2만큼 증가시킵니다.
민첩성은 플레이어의 정확도를 민첩성/3만큼 증가시킵니다.
체취는 플레이어가 NPC를 유학할 확률을 체취/2만큼 감소시킵니다

10. 중복을 제거하라
(Kill Redundancy)

- 중복은 악이며, 혼란을 야기합니다.

한 문서를 '주' 문서로 만들고, 다른 문서가 그 문서를 참조하게 하세요.

"전투 능력치.doc"

근력은 플레이어의 공격력을 근력/2만큼 증가시킵니다.
민첩성은 플레이어의 정확도를 민첩성/3만큼 증가시킵니다.
체취는 플레이어가 NPC를 유학할 확률을 체취/2만큼 감소시킵니다

"아이템.doc"

마력이 부여된 아이템을 착용할 경우, 플레이어의 능력치가 증가할 수 있습니다. 자세한 것은 전투 능력치.doc를 참고하세요.

11. 애매한 표현 금지
(No Weak Language)

금지!

플레이어는 이성 NPC에게 구애할 수 있을지도 모릅니다.
훗날에는, 낭만적인 사랑의 노래를 불러서 여성을 꼬실 확률을 높힐 수 있는 기능을 추가할지도 모릅니다.
이게 구현이 되면, 아마도 플레이어들은 자신의 사랑 노래를 작곡할 수도 있습니다.

11. 애매한 표현 금지
(No Weak Language)

이게 더 좋습니다!

NPC와 연애하기 (프로토타입)

플레이어는 대화를 통해서 이성 NPC를 꼬실 수 있습니다

또한 플레이어는 자신이 배운 노래를 불러줌으로써, 이성 NPC를 꼬실 수 있습니다.

연애 고급 (4단계)

플레이어는 연애 시스템에서 사용할 자신만의 노래를 만들 수 있습니다.

11. 애매한 표현 금지
(No Weak Language)

- 명확한 평서문을 사용하세요.
  - 금지어: “아마도”, “가능할 수도” 혹은 “그럴지도 모릅니다”
  - 심지어 “할 수도 있습니다”도 피하세요.
- 여러가지로 해석가능한 표현을 쓰지 마세요.
- “우리”라는 말을 쓰지 마세요.
- 기획에서 방향을 명확히 선택하세요
- ‘아마도’에 해당하는 피처는 개발 후반으로 미루세요.

12. (결정의) 이유를 포함시켜라
(Capture your Reasoning)

- 그러나 구분해 두어라.

ㅋㅋㅋㅋ!

플레이어는 아이템을 땅에 떨어뜨릴 수 없습니다. 이렇게 하면 지저분한 것들이 눈에 않고, 수백 개의 아이템들이 걸리적 거리지 않게 해줄 겁니다.

12. (결정의) 이유를 포함시켜라
(Capture your Reasoning)

- 그러나 구분해 두어라.

이게 훨씬 더 좋습니다!

플레이어는 아이템을 땅에 떨어뜨릴 수 없습니다.

…

자주 질문하는 것들: 왜 플레이어는 아이템을 땅에 떨어뜨릴 수 없나요?

이렇게 하면 지저분한 것들이 눈에 않고, 수백개의 아이템들이 걸리적 거리지 않게 해줄 겁니다.

12. (결정의) 이유를 포함시켜라
(Capture your Reasoning)

- 이유를 포함시키는 것은 장기 프로젝트에 유용한데, 그 이유는 왜 그렇게 하기로 했는지를 말 그대로 종종 잊어버리곤 하기 때문입니다.
- 더 나아가, 이유를 포함시키면 같은 이슈에 대해서 논쟁이 재발하는 횟수를 줄여줍니다.

리드들을 위한 조언들
(Tips for Leads)

1. 점진적이고 반복적인 기획을 수용하라
(Embrace Iterative Design)

- 바로 다음에 올 개발 단계를 아주 상세하게 기획하세요.
- 다른 개발 단계는 맨-먼스 수준으로 간단하게 기획하세요.
- 한참 뒤에 개발할 피처에 너무 집착하지 마세요.
- 기획이 바뀌고, 반복적으로 개발될 때마다,
  문서를 재검토하세요.

2. 검색가능하게 하라
(Make it Searchable)

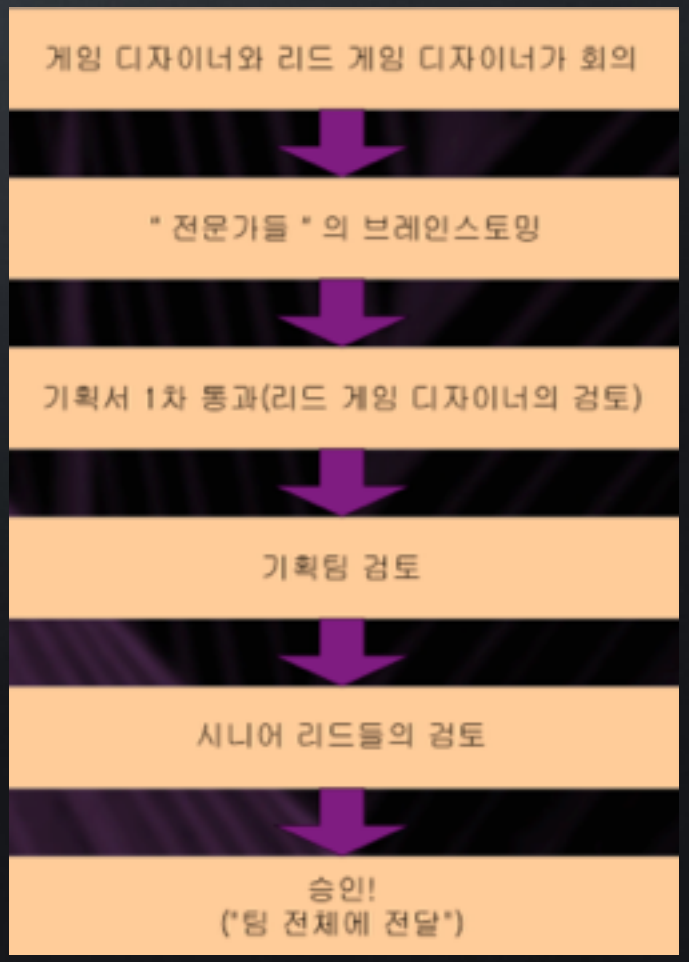
- 기획서는 개발자들이 필요한 것을 찾을 수 있을 경우에만 자료로서 가치를 가집니다.
- 추천하는 방법들:
  - 위키(Wiki)역자주
  - 데스크탑 검색(Desktop Search)
  - 기획 바이블(Design Bibles)역자주

3. 가능한 자동화하라
(Automate what you can)

- 증거가 필요하신가요?
  - Thottbot, Wowhead, Allakhazam[역자주]
- 문서 자동화의 장점들:
  - 정확도(특히, 포스트스크립적으로)
  - 검색 가능
  - 감사(auditing)나 보고서를 추가하기 쉬움

4. 기획서는 협업의 산물이다
(Design Documentation is a collaborative Process)

외부의 비평없이 허공에서 쓰여진 기획서는 '적과의 접전'에서 거의 살아남지 못합니다.



5. 언제나 개시 회의로 시작하라.

(Always start with a Kickoff Meeting)

- 게임 디자이너들이 리드 게임 디자이너를 만나서,
  다음 3가지 질문에 대답을 합니다:
  - 이 시스템의 목표는 무엇입니까?
  - 이 문서가 반드시 답변해주어야 하는 질문들
    (담고 있어야 하는 내용-역자주)은 무엇입니까?
  - 이 시스템이 얼마나 더 복잡해질 수 있을까요?

개시 회의
(The Kickoff Meeting)

- "이 시스템의 목표는 무엇입니까?"
  - 해당 시스템의 존재 가치를 찾습니다.
  - 울타리 기둥(fencepost) 문제역자주를 결정하도록 도와줍니다.
- 예시: 두 목표들은 가치가 있지만, 사전에 충분히 고려하지 않을 경우 서로 충돌됩니다:
  - "아이템 제작은 시간을 때우기 위한 부업으로서, 필드에서도 할 수 있다."
  - "대장장이는 작업장과 대장간을 소유할 수 있고, 다른 플레이어를 도와줌으로써 부와 명성을 얻을 수 있다."

개시 회의
(The Kickoff Meeting)

- "이 문서가 반드시 답변해주어야 하는 질문들은 무엇입니까?"
  - 결국 모든 시스템들은 서로 연관되어 있기 때문에, 이 문서가 어디까지만 다룰 것인지[역자주]를 결정하는 것이 중요합니다.
  - 리드들(Leads)이 문서화를 일정에 포함시키도록 합니다.
  - 졸속 기획(jumping the gun)을 방지합니다.
  - 다른 시스템이 해야 할 일을 가로채는 일(claim jumping)을 방지합니다.
  - 해당 피처(feature)[역자주]의 첫 버전에 집중합니다.

개시 회의
(The Kickoff Meeting)

- "얼마나 더 복잡해질 수 있을까요?"

  - **단순한 표현**(Token Representation). 우리는 텍스트 상자에 글머리 기호 기능을 추가하길 원합니다.
  - **경쟁력을 갖출까**(Competitive). 우리는 1등이 하고 있는 것을 약간 개선하길 (minor tweaks) 원하지만, 너무 위험한 선택을 하길 원치는 않습니다.
  - **대안이 될까**(Alternative). 엄청 크진 않지만, 경쟁자와는 확연히 다르게 합니다.
  - **혁신적이어야 하나**(Innovative). 우리는 이 피처를 구현해서, 경쟁자들을 밟아 버리고, 그들의 연인이나 부인의 통곡을 들을 것입니다.

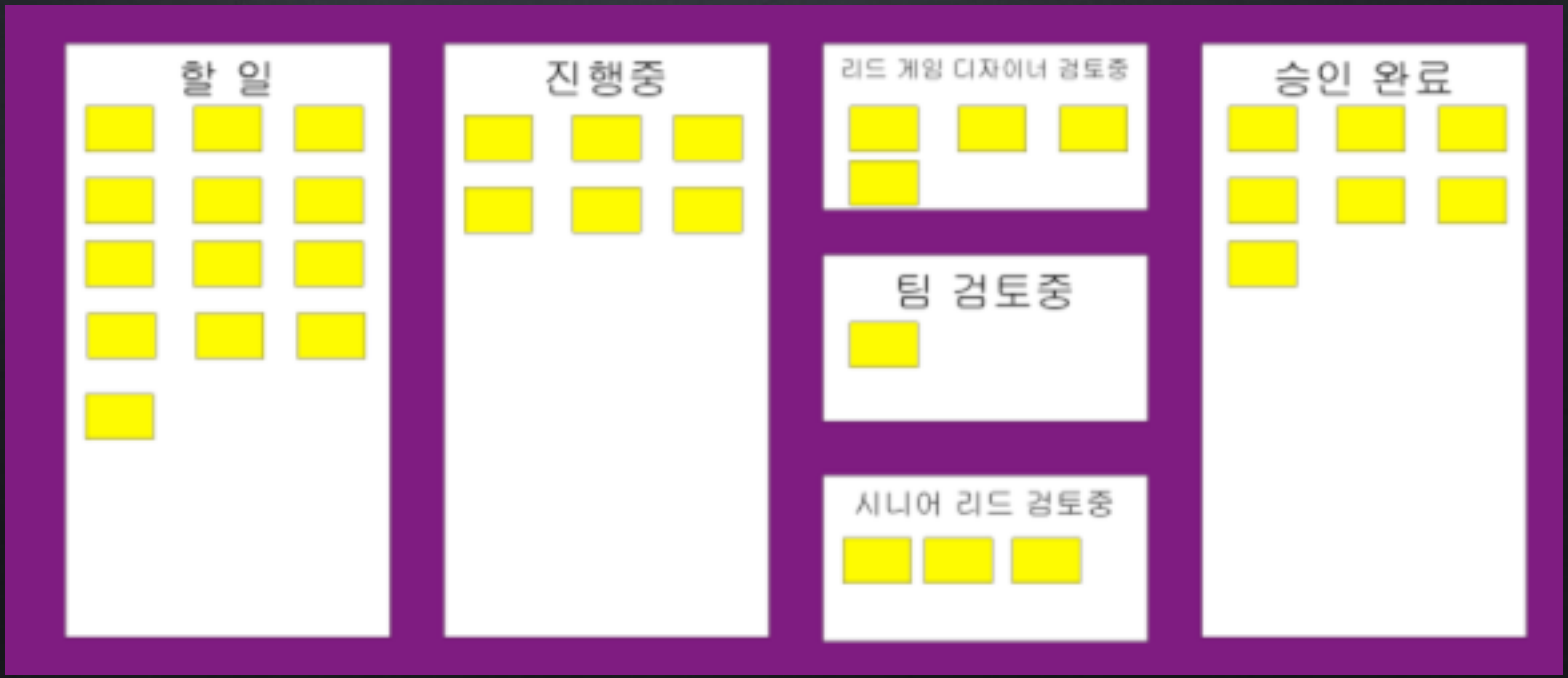
6. 승인 과정을 거쳐라
(Have an Approval Process)

- 기획을 단계별로 걸러내세요[역자주]
  - 리드 게임 디자이너가 먼저 승인합니다.
  - 그 다음에 기획 부서가 승인합니다.
  - 마지막에는 시니어 리드들/다른 부서들(Senior Leads/Cross-Team)이 승인합니다.
- 이렇게 하면, 기획 부서가 완성된 기획에 대해서 하나의 목소리를 낼 수 있게 됩니다.
- 항상 새로운 절차를 도입하고 운영하는 것은 어려운 일이지만, 한 번 탄력을 받고 나면 동료들이 가치를 인정하게 될 겁니다.

7. 전문가와의 상담을 의무화하라
(Mandate expert consultation)

- 게임 디자이너가 (주변 사람과 상의 없이) 고립된 상태로 문서를 작성하는 것을 금지하세요. 반드시 내부 전문가를 찾아야 합니다.
  - 팀 내의 다른 게임 디자이너들.
  - 해당 피처를 구현할 프로그래머들.
  - 특수한 전문 지식/기술을 가진 아티스트나 프로그래머
  - 도구에 대한 문서라면, '(그 도구의) 사용자들'
  - 다른 프로젝트의 개발자들(그들의 통찰이 특별히 가치있을 경우).

8. 진척 상황을 시각화하라
(Have a visual Method of Tracking Progress)



(저는 포스트-잇을 좋아합니다)

9. 변경 승인 절차를 만들어라.
(Have a change Process)

- 게임이 반복적으로 개발되면서, 기획이 바뀌게 됩니다. 팀의 의사결정자들에게 이 변경사항들을 알리는 절차가 필요합니다.

- 수석 리드들의 승인이 필요한 변경 사항이 발생했을 때, 보통 리드 게임 디자이너가 중재자가 됩니다.

10. 가끔 프로세스를 재검토하라
(Occasionally audit the process)

- 기획 문서화 절차는 반드시 팀에게 효과가 있어야 합니다. 만약 팀이 문서화 절차가 억압적이라고 여긴다면, 그 절차는 결국 폐지될 겁니다.
- 다음을 절대 잃어서는 안됩니다:
  - 간단 명료
  - 항상 최신을 유지
  - 프로그래머 친화적
- 4-6주마다, 자신(혹은 동료 프로그래머들)에게 어떤 게 효과가 있거나 없는지 물어보세요.

질문이 있으십니까?

부록:
GDC 2007에는 있었지만,
GDC 2008의 앵콜 강연에서는 사라진 슬라이드들

아이디어
(The Idea)

5. 늘 연구하라
(Research)

- 이전 게임들을 살펴보세요.
  - 특히, 실패작들을!
- 게임이 아닌 분야의 정보들도 살펴보세요.
  - 만약 요리 게임을 만들고 있다면, 요리의 철인(Iron Chef)역자주를 시청하세요!
- 해당 분야의 전문가와 만나보세요.
  - 다른 팀이나 다른 회사에 있는 분들이라도요!

6. 브레인스토밍하라
(Brainstorm)

- 많은 브레이스토밍 기법들을 모두 같은 일반적인 원칙들을 공유하고 있습니다:
  - 여러 명의 참가자들(6-8인).
  - 틀에서 벗어나서 생각하길 권장하세요.
  - 나쁜 아이디어란 없습니다.
  - 사람들의 공감을 얻은 아이디어를 찾아내세요.

7. 약점을 제거하라
(Cull the Weak)

- **이제**, 어떤 아이디어는 나쁜 아이디어입니다.
- 제거할 아이디어들:
  - 구현 불가능한,
  - 너무 야심찬,
  - (게임/목표)와 일치하지 않거나, 서로 상충하는,
  - 그냥 말그대로 이상한 아이디어들.
- 개시 회의(kickoff meeting) 때, 도출된 결론들을 기준(your razor)으로 사용하세요.
- 아주 흥미로운(intriguing) 아이디어는 어딘가 창고 같은 곳에 보관하고 싶을 수도 있습니다.
- 살아남은 아이디어들에 우선순위를 매기기 시작하세요.

8. 단순하게 만들어라
(Keep it simple)

- 최소한 처음에는 그렇게 시작하세요.
- 핵심에 집중하고, 가능한 빨리 플레이테스트가 가능한 것을 만드는 데 힘을 기울이세요.
- 피드백을 수렴해가며 반복적으로 기획하다보면 기획이 점점 복잡해질 겁니다.

만약 다른 사람들이 여러분의 기획서를 읽고 뭐라고 말할까 두려워하고 있다면, 그것은 분명 기획서가 너무 복잡하거나 너무 이상하기 때문일 겁니다.

9. 반복적인 개발을 수용하라
(Embrace Iteration)

- **어떤 게임 기획도 현실과 부딪혀서 상처입지 않고 살아남을 수 없습니다.**
- 게임 기획의 어떤 부분에도 미련을 갖지 마세요.
- 일단 여러분의 기획이 구현이 되면, 다시 갈아엎을 준비를 하십시요(prepare to iterate).

프로세스
(The Process)

1. 개시 회의를 의무화한다
(Enforce the Kickoff Meeting)

- 3가지 질문:
  - 목표는 무엇인가?
  - 이 기획서가 다루어야 할 범위는 어디까지인가?
  - 이 피처의 수용가능한 야심적인 목표는 무엇인가?
- 목표는 잘 정의된 방식으로 '상자의 경계'를 정함으로써, 게임 디자이너들에게 자율권을 부여해줍니다.

9. 아이스 께끼를 조심하세요
(Be Prepared to drop Trou)

- 언제 외부인들이 기획서를 보고 싶어할지 미리 알 수가 없습니다.
- 문서는 반나절 정도 미리 알려주면 제공가능해야 합니다.
- (혼란을 야기시키실 수 있으므로 기획에 대한) 기획팀 내부의 갈등이 기획서에서 드러나지 않도록 하거나, 손쉽게 삭제할 수 있어야 합니다.

결코 외부에 알려서는 안되는 게 있는지,
반드시 사전에 프로듀서들과 확인하세요

10. 유효 기간을 고려하라
(Consider expiration Dates)

- 길을 잃은 아이디어들:
  - 모든 문서는 6개월까지만 유효하며, 그 다음에는 게임 디자이너들이 그 문서들을 재검토하고 아직도 유효한지 확인해야 합니다
    - 기획상의 변경을 확인합니다
    - 우선순위의 변경을 확인합니다
    - 구현된 기획이 있는지 확인하고, 제대로 구현되었는지 확인해줍니다
- 검토해야 할 문서가 많을수록 더 힘듭니다

문서를 최신 상태로 유지할 필요가 있을까?
(Does your documentation need to stay up to date?)

- 여러분의 상황에 따라 다릅니다:
  - 대형 프로젝트에는 필요합니다.
  - 긴 수명을 가진 게임들(라이브 서비스나 확장판)이나 이직률이 높은 팀들도 마찬가지입니다.
  - 만약 외부 파트너들이 자꾸 귀찮게 군다면, 역시 필요합니다.

스크럼과 문서화
(Scrum and Documentation)

짤막한 소개(Brief Overview)

- 여러 분야의 전문가들로 구성된 작은 팀들
- 자신의 운명을 스스로 관리
- 구현할 피처들과 세부 피처들로 구성된 제품 백로그
  - 제품 책임자(보통 프로듀서나 수석 게임 디자이너)가 우선순위를 결정
  - 핵심은 팀은 항상 가장 중요한 것을 먼저 작업한다
- 스크럼은 '가벼운 문서화'를 지향
  - 심지어 소수의 지지자들은 '문서 자체를 만들지 말아야 한다'고 주장

그래서 어떻게 바뀌었을까?
(So what does this change?)

- 많이 놀라울 정도는 아님
  - 아직도 배급사와 외부 계약 파트너를 만족시키기 위한 문서화가 필요
  - 아직도 QA 테스트 계획수립을 위한 문서화가 필요
  - 아직도 아이디어를 산출하는 절차가 필요
- 촛점의 변화
  - 반복적인 개발과 사후-구현이 중요해짐.
  - 비-프로그래머 독자들이 중요해짐.
- 가장 중요한 변화
  - 문서화에 사용자 스토리를 사용

- 번역
  - 김기웅 (http://twitter.com/KayKimTwit)
- 편집
  - 정진호 (http://www.facebook.com/jinho.jung)
- 원문 출처:
  - GDC 2008 & GDC Vault: http://goo.gl/Y1780